

## **CHAPTER FOUR: SYSTEM ANALYSIS, DESIGN, AND IMPLEMENTATION**

### **4.1 Introduction**

This chapter presents the system analysis, design, and implementation of the E-Supervisor Platform, a digital platform developed to enhance the management and tracking of undergraduate research projects at the University of Kigali (UoK). The system is designed to improve efficiency, accessibility, and transparency in academic supervision operations by replacing traditional manual and isolated communication methods such as face-to-face meetings, scattered email threads, and paper-based records with a centralized, web-based solution.

The proposed system enables different users (students, faculty supervisors, and Heads of Departments) from the School of Computing and Information Technology (SCIT) to interact through a unified platform. It supports core functionalities such as student-supervisor allocation management, electronic research proposal and chapter document submission, automated workflow notifications, milestone progress tracking, and reporting analytics for departmental oversight.

The system was developed utilizing an Agile software development methodology and backed by a centralized Database Management System (DBMS) to translate manual academic workflows into structured relational tables. This technological approach ensures security, role-based access control, and efficient data handling while directly addressing the administrative boundaries and limitations of the existing supervision process.

### **4.2 Data Analysis and Presentation**

Data analysis was conducted to assess the performance of the existing manual research supervision workflows used at the University of Kigali and to identify key operational challenges requiring structural improvement. Data was collected through a multi-tool triangulation approach, including questionnaires, semi-structured interviews with supervisors and HODs, physical observations of student-faculty interactions, document reviews of university guidelines, focus group discussions, and online surveys.

The analysis focused heavily on how student projects are recorded, how chapter submission and feedback processes are managed, and how milestone progress information is tracked and shared with departmental leadership. The final sample size utilized for this analysis consisted of 265 respondents (250 final-year SCIT students, 13 faculty supervisors, and 2 Heads of Departments). The findings revealed that the current supervision environment operates through un centralized, manual channels, which severely limits collaboration, delays evaluations, and compromises institutional quality assurance.

#### **4.2.1 Weaknesses Observed in the Existing Supervision System**

The existing traditional project supervision system at the University of Kigali exhibits several systemic weaknesses due to its reliance on manual, fragmented, and paper-or-email-driven workflows:

- 1. Manual and Inconsistent Progress Recording**

Supervision milestones are not tracked on a standardized digital log. Instead, records are maintained via loose personal notes or disparate spreadsheets, leading to inconsistencies, data fragmentation, and an inability to maintain a clear audit trail of student progress.

- 2. Prolonged Feedback and Communication Turnarounds**

Relying on physical meetings and un centralized email boxes creates structural communication breakdowns. Students routinely encounter extensive delays in receiving iterative feedback and editing corrections, which stalls research progression and risks missing strict graduation timelines.

- 3. High Risk of Document and Version Loss**

Submitting chapter drafts through printed copies or scattered email threads introduces version control chaos. Important research documentation, reviewer comments, and stage approvals are highly vulnerable to physical loss, accidental deletion, or misplacement.

- 4. Lack of Real-Time Milestone Visibility**

The current system fails to provide real-time updates on active research statuses. Neither supervisors nor students can utilize a dashboard view to track completed versus pending tasks, making it difficult to systematically manage deadlines.

#### **5. Inefficient Supervisor Workload Management**

Without a central database platform, academic coordinators cannot audit active faculty review workloads dynamically. This results in highly imbalanced supervisor allocations, where some lecturers are overloaded with students while others remain underutilized.

#### **6. Limited Institutional Transparency and Accountability**

Students and supervisors frequently lack a transparent, shared system record of interactions, submitted drafts, or grading milestones. This absence of verification can lead to communication disputes and administrative oversights regarding evaluation fairness.

#### **7. Severe Difficulty in Generating Reports**

Compiling departmental supervision summaries, performance indicators, or identifying stalled student portfolios requires manual aggregation by project coordinators. This manual reporting process is slow, labor-intensive, and highly prone to reporting errors.

#### **8. Geographical and Schedule Boundaries**

The traditional process demands restrictive physical coordination. Busy academic schedules, supervisor workloads, and off-campus constraints create unnecessary access bottlenecks for remote student-mentor collaboration.

### **4.3 Interpretation of Findings / Results**

The findings from the data analysis conclusively indicate that the current manual supervision process at the University of Kigali is fragmented, operationally inefficient, and unable to support expanding student enrollments effectively. The total reliance on un centralized tools limits

visibility into the research lifecycle, slows down necessary academic feedback loops, and places undue administrative stress on both faculty supervisors and departmental coordinators.

These challenges directly validate the theoretical framework of the Technology Acceptance Model (TAM) used in this study, which highlights that users reject manual workflows when they are perceived as unnecessarily difficult and counterproductive to academic performance. The high dissatisfaction rates with document tracking errors and communication gaps underscore a critical demand among students and staff for an intuitive, automated platform.

Consequently, these results highlight the critical functional requirement for a dedicated, centralized digital solution like the proposed **E-Supervisor Platform**. The system directly addresses these documented institutional weaknesses by introducing secure role-based dashboard control, centralized relational database archiving to eliminate document loss, automated deadline notifications, and live progress reporting. Ultimately, transitioning to this automated workflow will eliminate data bottlenecks, maximize administrative transparency, and significantly improve undergraduate research completion rates at the University of Kigali.

#### **4.4 Summary of Findings**

The analysis of the existing research supervision workflows at the University of Kigali (UoK) revealed several critical operational challenges mainly caused by a total lack of integration and a heavy reliance on manual or isolated digital processes. The study found that un centralized communication methods such as physical face-to-face meetings, loose paper records, and fragmented personal email threads limit real-time oversight, leading to structural delays in iterative feedback and inefficient utilization of academic faculty resources.

Additionally, the absence of a centralized database management system results in prolonged feedback turnaround times when reviewing chapters, severe difficulties in tracking specific student project milestones, and structural challenges in maintaining accurate departmental records. The lack of a shared, real-time tracking interface reduces administrative transparency, making it exceptionally hard for undergraduate students to know their precise review status and for project coordinators or Heads of Departments (HODs) to monitor faculty workloads effectively. Structured interaction and workflow accountability between the independent stakeholders are

minimal, as there is no unified platform for archiving editing corrections or transmitting immediate status updates.

Overall, the findings demonstrate that the current traditional methods are highly inefficient, fragmented, and unable to support academic excellence or guarantee timely student graduation metrics. These challenges provide a strong justification for the development and implementation of the E-Supervisor Platform, which aims to centralize institutional data, eliminate document-tracking errors, maximize feedback transparency, and provide secure, real-time workflow information sharing among students, supervisors, and department heads.

#### **4.5 Description of the Existing System**

The existing project supervision workflows within the School of Computing and Information Technology (SCIT) at the University of Kigali operate independently, relying almost entirely on manual interventions and standalone, uncentralized channels. Final-year students must physically coordinate with their assigned faculty supervisors or send chapter drafts through personal email threads, while supervisors and department heads manage student records using paper registers or simple spreadsheet logs.

Academic project information such as assigned research topics, completed milestones, and reviewer corrections is recorded separately across different faculty personal notes. When a student needs feedback on a research proposal or draft chapter, they must wait for physical appointments or check their email continuously, as there is no central system to flag pending corrections or track review queues. Document submission and subsequent approval status updates are recorded either in notebooks or basic local documents, which do not automatically update the department or notify project coordinators in real time.

Because all supervision logs are stored locally by individual supervisors, it is nearly impossible for university coordinators to share resources, verify supervisor workloads, or retrieve historical data across the active student portfolio. Departmental report generation, such as tracking delayed projects, calculating overall thesis pipelines, or verifying supervisor performance indicators, must be compiled manually, making the process highly labor-intensive and prone to reporting errors. Formal communication between students, faculty supervisors, and academic leadership remains

strictly limited, with no centralized repository to log interaction histories or version controls. As a result, the existing approach lacks operational efficiency, performance transparency, and scalable data control, especially in an environment experiencing rising student enrollments.

#### **4.6 Description of the New System**

The proposed solution is the E-Supervisor Platform, a centralized web-based information system designed to integrate and automate academic project supervision workflows at the University of Kigali. The system replaces isolated email threads and manual paperwork with a secure, digital platform that allows students, supervisors, and HODs to interact and manage research milestones in real time.

Students can securely log into their specific dashboards to submit research proposals, upload independent chapter drafts, check approval statuses instantly, and view structured feedback from their supervisors. Faculty supervisors can manage their assigned student lists, track completion milestones, review uploaded documentation, and input edit corrections directly into the system interface. Heads of Departments (HODs) oversee broad institutional performance indicators, assign students to faculty members based on availability, audit active review workloads, and instantly generate comprehensive departmental progress reports.

Each interaction within the platform captures critical metadata, including user authentication credentials, document submission timestamps, supervisor review dates, and milestone completion statuses (e.g., Pending, Under Review, Approved, Rejected). The system automatically updates the student's milestone track and updates a central audit trail to eliminate communication disputes.

All operational data is managed securely by a centralized Database Management System (DBMS), allowing real-time data access, automated reporting, and accelerated evaluation turnarounds. Automated workflow notifications and dashboard reminders are triggered dynamically when documents are submitted or when a draft remains unreviewed past university response deadlines, dramatically enhancing communication efficiency and institutional accountability. By centralizing these distinct administrative phases into a single secure application, the platform removes tracking bottlenecks and optimizes graduation success rates.

#### 4.6.1 Functional Requirements

The functional requirements define the specific core capabilities, inputs, and operational tasks that the E-Supervisor Platform must perform:

**User Authentication and Authorization:** The system must support secure user registration and role-based login interfaces for students, faculty supervisors, and HOD administrators.

**Supervisor Allocation Management:** The system must allow HODs to dynamically assign students to faculty mentors and balance active supervisory workloads.

**Electronic Document Submission:** The system must enable students to upload research proposals and chapter drafts in structured digital formats.

**Digital Feedback Mechanism:** The system must allow assigned supervisors to review submissions, insert comments, and select clear milestone approval or rejection states.

**Real-Time Milestone Tracking:** The system must track active research progress and display completed versus pending chapters on visual dashboards.

**Automated Workflow Notifications:** The system must automatically trigger live dashboard updates and alerts when a student uploads a file or when a supervisor completes a review.

**Dynamic Report Generation:** The system must compile real-time statistics and generate reports on departmental thesis pipelines, supervisor workloads, and stalled projects.

**Data Control and User Management:** The system must allow administrators to manage user accounts, update database records, and modify baseline system parameters.

#### 4.6.2 Non-Functional Requirements

The non-functional requirements define the quality attributes, performance criteria, and operational constraints of the platform:

**Performance:** The system must respond quickly to user actions, such as document uploads, milestone updates, and report queries. Page loading and database lookups should complete within a few seconds to ensure efficient user interaction.

**Availability:** The platform must maintain high operational uptime (minimum 99.9%), ensuring that off-campus students and busy faculty members can securely access research tracking features at any time.

#### **Reliability and Data Integrity:**

The database must enforce strict relational constraints to ensure all tracking information is accurate, normalized, and consistent.

All database transactions must follow strict atomic principles, ensuring entries are either completely written or rolled back in full to prevent data corruption.

The system must perform regular, automated database backups to safeguard student research documentation against physical infrastructure failures.

Secure recovery protocols must be in place to restore data integrity swiftly in the event of a system crash.

**Maintainability:** The platform must be engineered using a modular system architecture, allowing the technical team to update components or integrate future university systems without disrupting ongoing operations.

**Security and Compliance:** The system must implement strict role-based access control (RBAC). Student research data, reviewer critiques, and departmental files must only be accessible to explicitly authorized users based on verified system roles.

#### **4.6.3 System Configuration**

The system configuration outlines the baseline hardware environments and software stacks required to develop, deploy, and execute the E-Supervisor Platform effectively.



#### 4.6.3.1 Hardware Specifications

**Client Devices:** Personal computers, laptops, or web-enabled smartphones for students, faculty supervisors, and HOD administrators.

**Processor (Server/Client):** Minimum Intel Dual-Core or modern equivalent processor.

**Memory (RAM):** Minimum 4GB RAM on client devices; 8GB or higher recommended for the hosting application server.

**Storage:** Minimum 100GB of secure Solid State Drive (SSD) storage allocated on the application/database server to comfortably archive student documentation.

**Internet Connectivity:** A stable internet connection is mandatory to support real-time web portal access and live database synchronization.

#### 4.6.3.2 Software Specifications

**Operating System:** Compatible across Windows 10/11, Linux (Ubuntu), or macOS environments.

**Backend Technologies:** python runtime environment and fast API.

**Frontend Technologies:** React.js library combined with HTML5, CSS3, and TypeScript for responsive, interactive user dashboards.

**Database Management System (DBMS):** PostgreSQL Relational Database Server for managing structured tables, tracking logs, and foreign key relationships.

**Development & Version Tools:** Visual Studio Code (IDE) and Git/GitHub for collaborative version control.

#### 4.7 Illustration of the New System / Solution

The proposed E-Supervisor Platform operates through a structured, automated digital workflow that coordinates the actions of students, supervisors, and department heads within a single secure environment.

The process begins when an authorized student logs into the web portal and accesses their personalized dashboard to upload a new research proposal or chapter draft. The system registers the file in the database, establishes a precise timestamp, and immediately marks the milestone status as Pending Review. Concurrently, an event-driven system trigger generates a notification on the assigned supervisor's dashboard alerting them to the new submission.

The supervisor reviews the uploaded research documentation directly from their screen and inputs specific editing corrections or text feedback into the system. The supervisor then selects a decision state: if rejected, the status changes to rejected, prompting the student to upload an updated version; if accepted, the status changes to Approved, moving the student to the next sequential chapter milestone.

The system continuously logs these step-by-step completions, updating the institutional timeline automatically. If a submission remains unreviewed past the department's standard timeframe, the platform flags the delay count down of days on the dashboard to ensure supervisor accountability. HOD administrators utilize their high-level interface to monitor these tracking activities across the entire department, balance faculty workloads, and instantly pull accurate compliance reports. This integrated workflow ensures seamless collaboration, eliminates physical document loss, and maximizes performance transparency.

#### **4.7.1 Data Flow Diagram (DFD)**

A Data Flow Diagram (DFD) visually represents the architectural movement of information within the E-Supervisor Platform. It maps out how tracking data is entered by users, processed by backend application routines, stored securely within relational database tables, and retrieved to update interactive dashboards, providing a clear map of the platform's underlying mechanics.

The diagram models the explicit exchange of information between the key external entities (Students, Supervisors, and HODs) and the core processing engine:

**Student Data Flow:** Students input user credentials, research topics, and chapter files into the system; the processing block returns live submission statuses, historical reviewer feedback, and milestone completion parameters.

**Supervisor Data Flow:** Supervisors interact with the system by pulling student document records and submitting evaluation keys, approval updates, and text corrections; the system returns filtered student lists, active workload analytics, and pending review alerts.

**HOD Data Flow:** HOD administrators input user registration details, student-supervisor allocation maps, and query parameters; the system processes these inputs against the database to return dynamic supervision metrics, dashboard graphics, and comprehensive evaluation summaries.

In addition, the DFD details how information transitions from user-side entry components to the Express backend framework, which routes the requests to be written or read within the central PostgreSQL database. Once updated, the data flows back to the user interfaces, generating real-time output views such as updated milestone graphics, audit charts, and live notification triggers. This ensures that all data transactions remain logically connected, maintaining a secure and highly reliable information system for the University of Kigali.

#### **4.7.1.1 Components of Data Flow Diagrams.**

Data Flow Diagrams (DFDs) are composed of four basic symbols that represent how data moves and is processed within the E-Supervision platform.

##### **External Entity**



Figure 1 external entity

External entities are represented by rectangles or squares. They are actors or system users that interact with the system by providing input data or receiving output information.

These entities act as sources or destinations of data but do not process data themselves.

## Process



Figure 3 process

Processes are represented by circles, ovals, or rectangles. A process transforms incoming data into meaningful output.

Each process takes input data, performs operations, and produces output data.

### 1. Data Flow



Data flow is represented by arrows. It shows the direction and movement of data between entities, processes, and data stores.

Data flow helps in understanding how information moves throughout the system.

### 2. Data Store

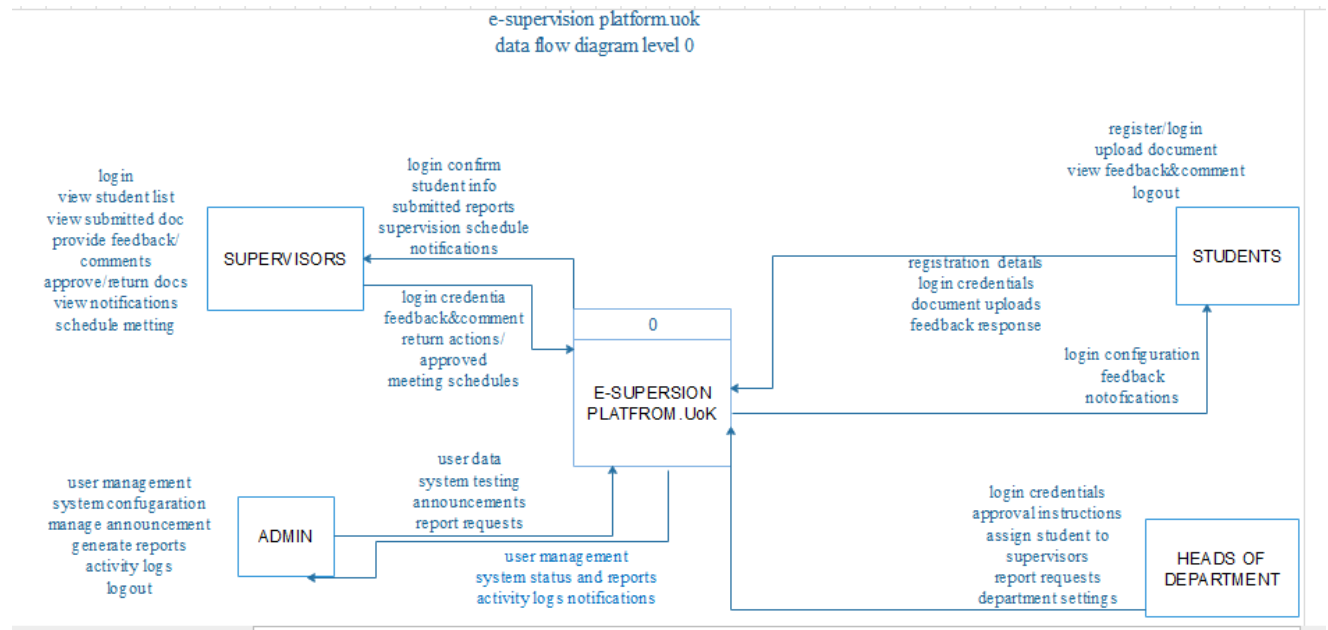


Data stores are represented by two parallel horizontal lines. They indicate where data is stored within the system.

These four components work together to provide a clear and structured representation of how data flows within the **E-Supervision platform**, making it easier to design, analyze, and understand the system.

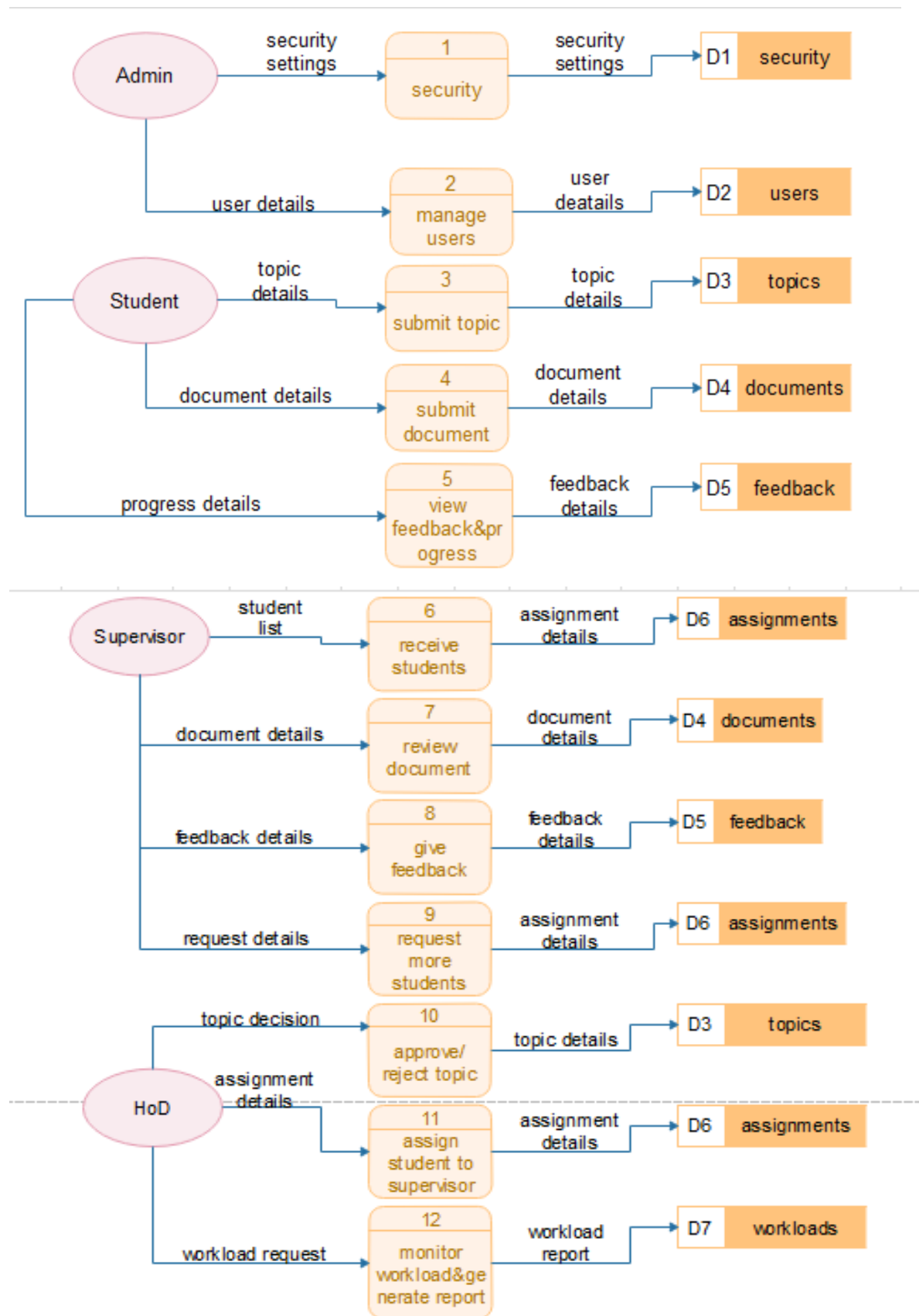
#### 4.7.1.2 Level 0 (Context Diagram)

The Level 0 Data Flow Diagram, also known as the Context Diagram, provides a high-level overview of the E-Supervision platform. It represents the entire system as a single process and illustrates how it interacts with external entities through major data flows.



#### 4.7.1.3 Level 1 of DFD

A Level 1 Data Flow Diagram (DFD) provides a more detailed representation of the E-Supervision platform compared to the Level 0 (Context Diagram). It breaks down the main system process into smaller sub-processes to show how data is processed internally.



#### **4.7.2 Database Normalization.**

Database normalization is a systematic data design technique applied to the E-Supervision platform's relational database to ensure that data is organized efficiently, redundancy is minimized, and structural relationships between entities are clearly defined. By transitioning the University of Kigali's manual undergraduate research management into a centralized, automated digital ecosystem, the database was rigorously normalized to improve data integrity, eliminate update anomalies, and guarantee secure, rapid storage and retrieval of institutional information including users, students, research chapters, milestones, supervisor allocations, and performance reports.

There are three main normal forms sequentially applied to the database architecture:

##### **First Normal Form (1NF)**

The primary objective of First Normal Form is to ensure data scalability and eliminate multi-valued or composite attributes within the tables. In the E-Supervision platform, each table is structured to satisfy the following rules:

**Atomic Fields:** Each field contains atomic (indivisible and single-valued) values only.

**No Repeating Groups:** There are no repeating groups, comma-separated values, or multi-valued attributes within a single row.

**Unique Identification:** Every row is uniquely identifiable by a defined primary key.

##### **Second Normal Form (2NF)**

The system satisfies Second Normal Form by addressing the relationship between the primary key and non-key attributes. To achieve 2NF, a table must first meet all 1NF requirements and ensure that:

**Full Functional Dependency:** All non-key attributes are fully functionally dependent on the primary key.

**No Partial Dependencies:** There are no partial dependencies, meaning no non-key attribute depends on only a portion of a composite primary key.

### **Third Normal Form (3NF)**

The database achieves Third Normal Form by eliminating structural dependencies that do not involve the primary key, thereby ensuring optimal data integrity during runtime updates. The platform satisfies 3NF because:

**Satisfies 2NF:** It meets all criteria for both First and Second Normal Forms.

**No Transitive Dependencies:** There are no transitive dependencies present, meaning non-key attributes do not depend on other non-key attributes.

#### **User Table:**

**Central account table for students, supervisors and HoDs.**

| Column    | Type           | Notes                        |
|-----------|----------------|------------------------------|
| id        | INTEGER PK     | Auto-increment               |
| full_name | VARCHAR        | Required                     |
| email     | VARCHAR UNIQUE | Staff login; student contact |
| password  | VARCHAR        | bcrypt hash                  |
| role      | ENUM           | STUDENT, SUPERVISOR, HOD     |



| Column              | Type           | Notes                        |
|---------------------|----------------|------------------------------|
| department          | VARCHAR        | IT, LAW, BUSINESS, EDUCATION |
| title               | VARCHAR        | Job title                    |
| program             | VARCHAR        | Academic programme           |
| phone               | VARCHAR        | Contact number               |
| avatar_url          | VARCHAR        | Optional profile image       |
| bio                 | TEXT           | Profile text                 |
| active              | BOOLEAN        | Default true                 |
| registration_number | VARCHAR UNIQUE | 12-digit student login ID    |
| created_at          | TIMESTAMP      | Account creation             |

### Projects Table:

**One capstone project per assigned student.**

| Column        | Type       | Notes                           |
|---------------|------------|---------------------------------|
| id            | INTEGER PK |                                 |
| title         | VARCHAR    | Project title                   |
| description   | TEXT       | Summary                         |
| current_phase | VARCHAR    | e.g. Chapter 3                  |
| status        | ENUM       | PROPOSAL ... COMPLETED, ON_HOLD |

| Column        | Type          | Notes               |
|---------------|---------------|---------------------|
| progress      | INTEGER       | 0–100               |
| start_date    | DATE          |                     |
| due_date      | DATE          |                     |
| student_id    | FK → users.id | Project owner       |
| supervisor_id | FK → users.id | Assigned supervisor |

### Tasks Table:

### Milestones and work items for a project.

| Column      | Type          | Notes                                     |
|-------------|---------------|-------------------------------------------|
| id          | INTEGER<br>PK |                                           |
| title       | VARCHAR       |                                           |
| description | TEXT          |                                           |
| category    | VARCHAR       | e.g. RESEARCH, DESIGN                     |
| status      | ENUM          | UPCOMING, IN_PROGRESS, COMPLETED, OVERDUE |
| priority    | ENUM          | LOW, MEDIUM, HIGH                         |
| progress    | INTEGER       | 0–100                                     |
| due_date    | DATE          |                                           |

| Column     | Type                | Notes                  |
|------------|---------------------|------------------------|
| milestone  | BOOLEAN             | Major deliverable flag |
| project_id | FK<br>→ projects.id |                        |

### Submissions Table:

**Student file uploads (PDF/Word)for supervisor review.**

| Column       | Type                | Notes                                                        |
|--------------|---------------------|--------------------------------------------------------------|
| id           | INTEGER<br>PK       |                                                              |
| title        | VARCHAR             | Submission title                                             |
| notes        | TEXT                | Student notes                                                |
| file_url     | VARCHAR             | e.g. /api/files/{stored_name}                                |
| file_name    | VARCHAR             | Original filename                                            |
| status       | ENUM                | SUBMITTED, UNDER_REVIEW, APPROVED, NEEDS_REVISION            |
| submitted_at | TIMESTAMP           | Used for <b>08:00–17:00 window</b> and <b>queue priority</b> |
| project_id   | FK<br>→ projects.id |                                                              |

| Column  | Type             | Notes                |
|---------|------------------|----------------------|
| task_id | FK<br>→ tasks.id | Optional linked task |

### **Feedback Table:**

**Supervisor comments on submission or general project feedback.**

| Column        | Type                | Notes              |
|---------------|---------------------|--------------------|
| id            | INTEGER PK          |                    |
| title         | VARCHAR             |                    |
| content       | TEXT                | Required           |
| created_at    | TIMESTAMP           |                    |
| project_id    | FK → projects.id    |                    |
| author_id     | FK → users.id       | Usually supervisor |
| submission_id | FK → submissions.id | Optional           |

### **Notifications Table:**

**In app alerts for deadlines, submissions and proposals.**

| Column | Type          | Notes |
|--------|---------------|-------|
| id     | INTEGER<br>PK |       |
| title  | VARCHAR       |       |

| Column      | Type             | Notes                                            |
|-------------|------------------|--------------------------------------------------|
| message     | TEXT             |                                                  |
| type        | ENUM             | DEADLINE, FEEDBACK, ASSIGNMENT, SYSTEM, APPROVAL |
| severity    | ENUM             | LOW, MEDIUM, HIGH                                |
| read        | BOOLEAN          |                                                  |
| action_path | VARCHAR          | Frontend route, e.g. /supervisor/reviews         |
| created_at  | TIMESTAMP        |                                                  |
| user_id     | FK<br>→ users.id | Recipient                                        |

### Topic\_proposals Table:

**Public capstone topic applications before a student portal account exists.**

| Column              | Type       | Notes               |
|---------------------|------------|---------------------|
| id                  | INTEGER PK |                     |
| full_name           | VARCHAR    | Applicant name      |
| email               | VARCHAR    | Contact email       |
| registration_number | VARCHAR    | 12-digit reg number |
| phone               | VARCHAR    | Optional            |

| Column                    | Type          | Notes                              |
|---------------------------|---------------|------------------------------------|
| program                   | VARCHAR       | Selected programme                 |
| department                | VARCHAR       | Routed to HOD                      |
| topic_1 ... topic_3       | VARCHAR       | Three proposed titles              |
| abstract_1 ... abstract_3 | TEXT          | Three abstracts                    |
| supervisor_choice_1_id    | FK → users.id | 1st preference                     |
| supervisor_choice_2_id    | FK → users.id | 2nd preference                     |
| status                    | ENUM          | PENDING, APPROVED, REJECTED        |
| selected_topic_index      | INTEGER       | 1–3 when approved                  |
| rejection_reason          | TEXT          | When rejected                      |
| assigned_supervisor_id    | FK → users.id | Set on approval                    |
| student_user_id           | FK → users.id | Portal account created on approval |
| created_at                | TIMESTAMP     |                                    |

### **Supervisors\_student\_requests Table:**

**Supervisors requesting an additional student from their HoD.**

| Column        | Type          | Notes                 |
|---------------|---------------|-----------------------|
| id            | INTEGER PK    |                       |
| supervisor_id | FK → users.id | Requesting supervisor |

| Column       | Type      | Notes                       |
|--------------|-----------|-----------------------------|
| message      | TEXT      | Request justification       |
| status       | ENUM      | PENDING, APPROVED, REJECTED |
| hod_response | TEXT      | HOD reply                   |
| created_at   | TIMESTAMP |                             |

#### ➤ 4.7.4 Entity Relationship Diagram (ERD)

An Entity Relationship Diagram (ERD) illustrates the logical relationships between entities in a system and demonstrates how data is structurally connected within the relational database. Based on the implemented database schema for the E-Supervision platform, the main entities include Users, Projects, Tasks, Notifications, Topic\_proposals, Supervisors\_student\_requests, Submissions, and Feedback.

The ERD represents how these foundational entities interact securely within the system to transition the University of Kigali's manual undergraduate research management into an automated digital ecosystem. It supports core academic operations such as role-based authentication, student-supervisor allocation, milestone progress tracking, and electronic document feedback delivery.

#### ENTITIES FOR ERD (E-SUPERVISION PLATFORM)

**Users attributes:**

{

**int id PK**

**string email UK**

**string registration\_number UK**

**enum role**

**string department**

**string password**

**datetime created\_at**

**}**

**Projects attributes:**

**{**

**int id PK**

**int student\_id FK**

**int supervisor\_id FK**

**enum status**

**int progress**

**date due\_date**

**}**

**Submissions attributes:**

**{**

**int id PK**

**int project\_id FK**



**string file\_url**

**string file\_name**

**enum status**

**datetime submitted\_at**

**}**

**topic\_proposals attributes:**

**{**

**int id PK**

**string registration\_number**

**string department**

**enum status**

**int supervisor\_choice\_1\_id FK**

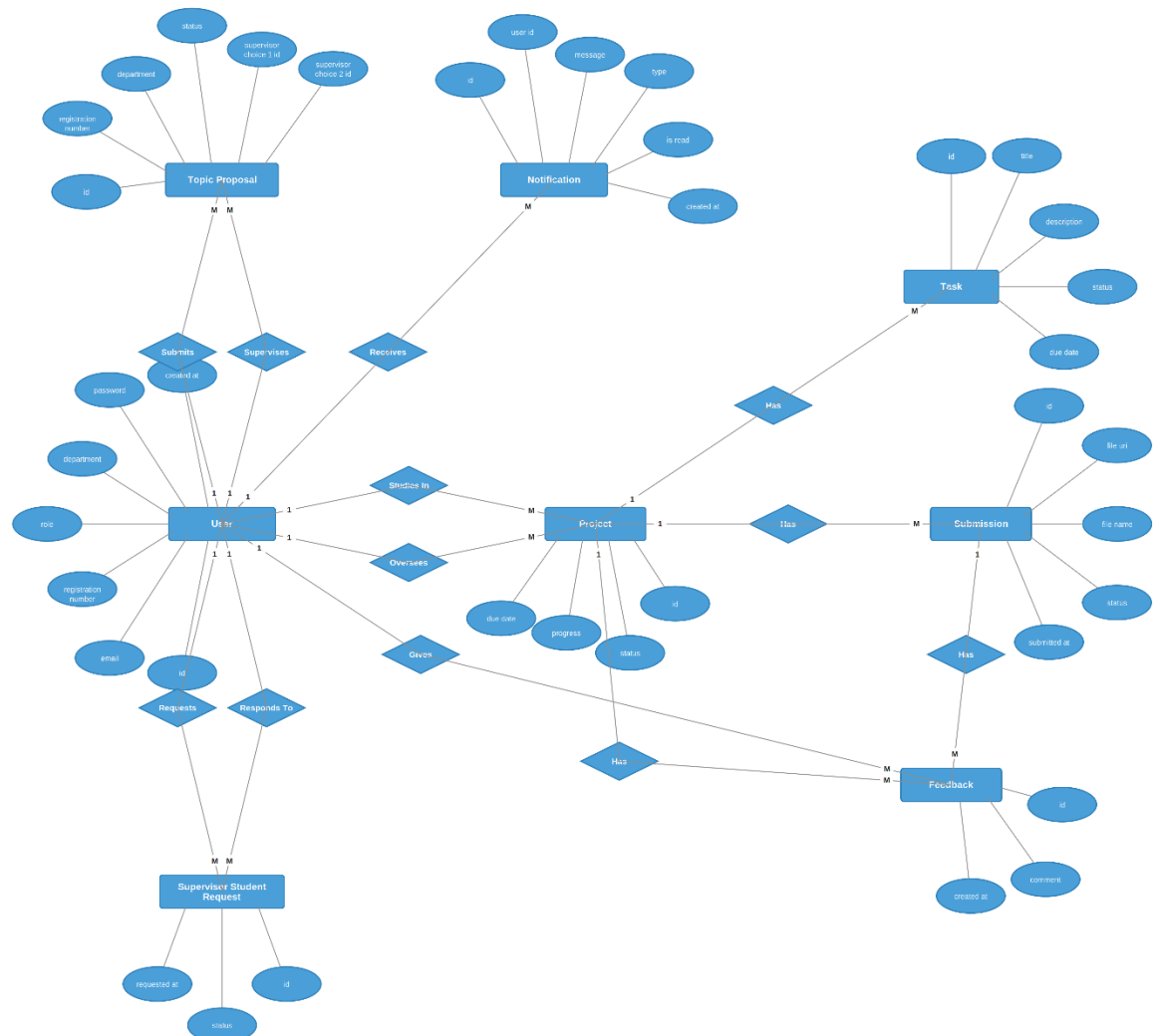
**int supervisor\_choice\_2\_id FK**

**int assigned\_supervisor\_id FK**

**int student\_user\_id FK**

**}**

## ENTITY RELATIONSHIP DIAGRAM



### 4.7.5 Physical Data Model

The Physical Data Model (PDM) of the E-Supervision platform represents how data is physically stored and implemented in the database. It defines the operational structure of tables, columns, specific data types, primary keys, foreign keys, logical constraints, indexing, and storage details within the PostgreSQL database management system.

The E-Supervision platform database is designed to support efficient storage, rapid retrieval, and automated management of data related to Users, Projects, Tasks, Notifications, Topic\_proposals,

Supervisors\_student\_requests, Submissions, and Feedback. Each entity identified in the logical engineering design is implemented as a table with clearly defined attributes, strict data types, and relational constraints to optimize performance for the School of Computing and Information Technology (SCIT).

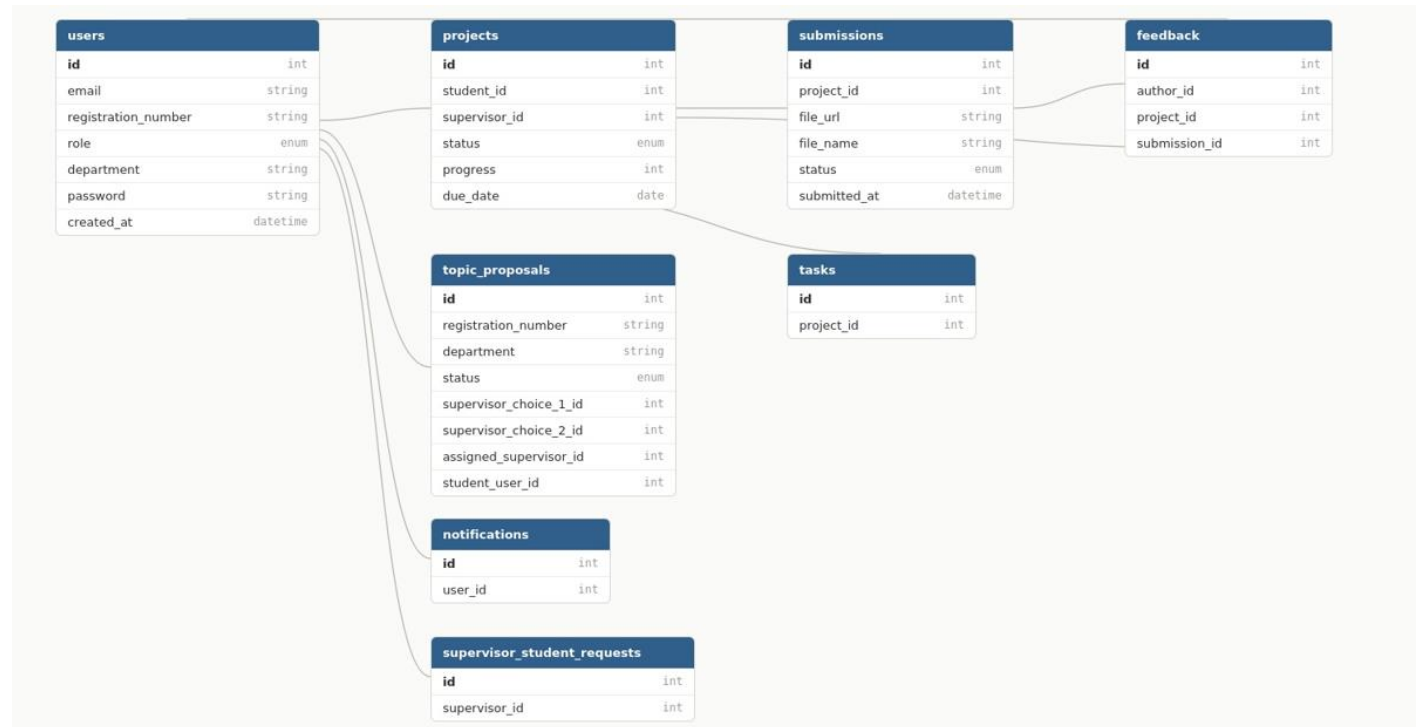


Figure 2 Physical Data Model

#### 4.9.3.2. Student Registration Form

The Student Login land page allows final-year undergraduate students within the School of Computing and Information Technology (SCIT) to access the platform using their existing institutional credentials. Instead of creating a new account, students log in directly by entering

their Registration Number (e.g., 2305000092) as their username, along with their pre-assigned Password.

Upon submission, the system validates these input parameters against structural formats and verifies them against the institutional database. Once securely authenticated, the system retrieves the student's profile data including their Full Name, Registration Number, and Institutional Email to grant them immediate access to their personalized dashboard. From this dashboard, students can manage their final-year requirements, submit milestone chapters, and request meetings with advisors.

#### **4.9.3.3. Supervisor Form**

The Supervisor Login Portal allows the 13 assigned faculty lecturers to access the platform directly using their existing institutional credentials. Rather than creating a new account, supervisors log in by entering their Institutional Email and their standard Password.

Upon submission, the system validates the format of the inputs and securely authenticates the credentials against the faculty database. Once verified, the system loads the supervisor's pre-existing profile details including their Full Name, Lecturer ID, and Specialization Field. This grants them immediate access to their dedicated workload tracking dashboards, where they can review uploaded student thesis chapters and submit structured evaluation feedback.

#### **4.9.3.4. Head of Department Form**

The **Head of Department (HOD) Login Portal** allows the head of department to access the platform using their institutional credentials. The HOD logs in directly using their **University Email** and standard **Password** without needing to register a new account.

Once authenticated, the HOD is granted access to a specialized administrative dashboard designed to manage and oversee the department's final-year project ecosystem. Through this dashboard, the HOD can:

**Allocate Research Resources:** Directly assign undergraduate students to available project supervisors.

**Balance Workloads:** Monitor active supervisor capacity in real-time to prevent individual faculty overloading.

**Track Operational Progress:** Oversee ongoing milestones, student submissions, and supervisor feedback across the department.

**Generate Analytics Reports:** Export structured progress reports and evaluation summaries for departmental review and university auditing.

#### **4.9.3.5. Admin**

The Admin/HOD Form allows system administrators and project coordinators to create and manage administrative profiles with full supervisory authorization permissions. It records descriptive profiles including Administrator Name, Coordinator ID, Email, and Password. This layer ensures secure identity mapping, enabling HODs to manage student-supervisor pairings, audit broad institutional performance indicators, and extract system analytics reports.

### **4.9. Implementation and Coding**

#### **4.9.1 Introduction**

The E-Supervision platform is a web-based, centralized digital solution designed to simplify, automate, and monitor undergraduate research projects within the University of Kigali's School of Computing and Information Technology (SCIT). The system aims to eliminate manual tracking bottlenecks, improve operational workflow transparency, and optimize collaboration by providing

automated features such as role-based user authentication, supervisor workload balancing, chapter submission version control, milestone tracking, and audit-ready feedback logging. This section details the practical implementation and coding aspects of the application, incorporating the chosen technologies, architectural design principles, normalized database schema, and core operational program modules.

#### **4.9.2 Description of Implementation Tools and Technologies**

The E-Supervision platform was engineered utilizing modern web technologies to achieve optimal scalability, robust security, and reliable performance during peak submission periods. The core stack selected for system construction comprises Python, Fast API, React.js, and PostgreSQL.

**Programming Languages & Backend Framework (Python & FastAPI):** Python serves as the primary backend programming language, powered by the FastAPI framework to construct a high-performance, robust RESTful API. This backend architecture implements core system validation rules, processes milestone status changes, coordinates secure file uploads, and manages real-time system alerts and user authentication.

**Frontend Library & Language (React.js & TypeScript):** The frontend is built using React.js paired with TypeScript to deliver a type-safe, component-based user interface. This combination ensures a highly responsive, interactive, and user-friendly web experience. TypeScript enforces strict data structures across the separate, customized views designed for students tracking milestones, supervisors auditing assigned workloads, and HODs allocating research resources.

**Database Management System (PostgreSQL):** PostgreSQL functions as the relational database management system to permanently house and organize all institutional data fields. Structured in accordance with Third Normal Form (3NF) design, it reliably maps data parameters for accounts, departments, student profiles, active supervisor allocations, document drafts, and interaction alerts while enforcing rigid foreign key constraints and transactional integrity.

#### **4.10 Testing**

#### **4.10.1 Introduction**

Testing represents a critical operational phase within the Software Development Life Cycle (SDLC) of the E-Supervision platform. It systematically verifies that the developed system operates perfectly in accordance with the specified functional and non-functional requirements. The core objective of testing in this project was to validate system execution accuracy, eliminate data handling errors, confirm role-based access rules, and guarantee that the system operates reliably for students and faculty within the SCIT department.

The structured testing framework focused closely on validating key institutional features, including:

User account registration and secure login authentication (Students, Supervisors, HODs, and Admins).

Automated and manual allocation mappings of students to specific faculty supervisors.

Digital transmission and storage validation of research chapter files.

Real-time submission status updating and version tracking.

Accurate recording and rendering of supervisor feedback corrections and meeting requests.

Compilation of departmental performance analytics and workload distribution sheets.

#### **4.10.2 Unit Testing Outputs**

Unit testing involves verifying the smallest isolated logic components of the system individually to ensure their underlying code executes flawlessly before broader integration. In the E-Supervision platform, unit testing evaluated individual software modules, internal database queries, and field validation functions built with PostgreSQL.

**Unit Testing Process:** Each isolated function block was executed independently against predetermined input metrics and boundary parameters. The unit testing scripts validated the following standalone modules:

**User Authentication Module:** Verifying password encryption routines, validation flags for invalid email strings, and correct session token issuing based on specific user roles.

**Allocation Module:** Testing the logic that updates supervisor assignment records and checks the capacity threshold values whenever an HOD pairs a student with a lecturer.

**Submission Module:** Verifying file extension constraints, validating storage directory routing, and testing file metadata insertions into the PostgreSQL schema.

**Feedback and Meeting Module:** Auditing the text block validation and confirmation triggers when logging a supervisor's inline comments or setting scheduling dates.

**Database Query Operations:** Isolating SQL insertion, update, retrieval, and deletion strings to ensure data updates execute cleanly without causing field corruption.

All unit testing evaluations were executed via standard development debuggers and manual code testing paths, confirming that individual application elements executed correctly.

#### **4.10.3 Validation Testing Outputs**

Validation testing was conducted to confirm that the completed platform satisfies user specifications and handles data appropriately under realistic operational scenarios. The platform was evaluated against practical administrative use cases within SCIT, confirming the following specific rules:

Ensuring users are rigidly restricted to features designated by their roles (e.g., validating that students are blocked from changing supervisor workload metrics or viewing other students' submissions).

Verifying that inputs strictly adhere to correct formats (such as institutional email validations and required numerical formatting for student registration numbers).

Confirming successful database commits and verified data retrievals for multi-part chapter uploads and milestone updates.



The outputs proved that the software application operates predictably, complies with field constraints, and achieves the required functional objectives.

#### **4.10.4 Integration Testing Outputs**

Integration testing was executed to verify that the separate, independently tested modules of the E-Supervision platform interact correctly when combined. This level of testing identifies data transmission faults, interface mismatches, or pipeline breakdown vulnerabilities across adjacent components.

During integration testing, the following core communication pathways were verified:

**Authentication and Access Control Integration:** Confirming that the login state securely feeds downstream modules, thereby permitting or denying access to supervisor workflows based on the verified role type.

**Submission Module and Database Integration:** Verifying that file upload events successfully trigger corresponding data rows inside the submissions table while pointing correctly to physical files on disk.

**Allocation Module and Profile Integration:** Ensuring that when an HOD modifies a supervisor allocation, the change instantly updates the supervisor's active workload counts and updates the student's assigned mentor profile.

**Feedback and Portal Alert Integration:** Confirming that when a supervisor saves correction notes, a corresponding dashboard notification indicator triggers automatically for the target student user.

Testing was systematically conducted using mock research data in a controlled staging environment, confirming smooth data flows across backend endpoints.

#### **4.10.5 System Testing Outputs**

System testing evaluated the E-Supervision platform as a fully unified, complete software asset to check total compliance against all integrated functional constraints and non-functional performance benchmarks.

Realistic end-to-end user scenarios were simulated simultaneously, such as a student user registering and uploading Chapter 1, an HOD assigning an active lecturer, and that lecturer receiving a dashboard notification, reviewing the draft, and saving a milestone approval state. System testing verified that the concurrent application paths execute reliably, maintain absolute data consistency across related tables, perform under load without latency, and preserve data confidentiality boundaries.

#### **4.10.6 Functional and System Testing Results**

The functional and system testing outcomes verified that the E-Supervision platform satisfies all software engineering constraints and operates reliably under standard deployment conditions. Functional testing verified that all core features including secure role-based authorization, supervisor pairing management, chapter document storage, milestone tracking, and text feedback logging produced accurate data states.

The application successfully achieved:

Dynamic validation of all user data inputs and rejection of corrupt parameters.

Complete restriction of unauthorized access to confidential supervisor dashboards.

High-speed storage, mapping, and retrieval of relational data records.

System testing confirmed that integrated components coordinate seamlessly without causing software lockups or data collisions. Performance checks indicated that the application processes simultaneous operations with minimal response delays, while security tests confirmed that student details and research assets remain securely isolated.

#### **4.10.7 Acceptance Testing Report.**

The Acceptance Testing Report summarizes the operational feedback gathered from selected stakeholders within the University of Kigali's School of Computing and Information Technology (SCIT) to verify that the platform fulfills institutional workflow requirements and user expectations.

Testing pathways were completed by actual undergraduate students, project coordinators (HODs), and faculty lecturers who performed real-life tasks, including:

Accessing individual portals using assigned credentials.

Uploading and tracking research chapter milestone files.

Reviewing submissions, inputting evaluation feedback, and scheduling meetings.

Processing supervisor allocations and extracting departmental progress reports.

Stakeholder feedback indicated that the E-Supervision platform is highly intuitive, accessible, and successful in reducing manual research management delays. Minor UI adjustments were introduced based on user feedback, and no critical code errors or data security failures were encountered. The acceptance testing results confirm that the software system fully satisfies user requirements and is ready for institutional implementation.

## **CHAPTER FIVE: CONCLUSION AND RECOMMENDATIONS**

### **5.0 Introduction**

This study focused on the development and implementation of the web-based E-Supervision platform aimed at optimizing undergraduate research project management and balancing faculty workloads through the deployment of modern digital technologies. As highlighted in Chapter One, traditional research supervision frameworks within many higher learning institutions, including the University of Kigali, frequently operate through manual, paper-based, or fragmented semi-digital processes, suffering from limited real-time progress tracking and communication bottlenecks.

These operational vulnerabilities result in significant delays in chapter draft retrieval, a lack of transparency regarding milestone approvals, severe data replication issues, and inequitable or opaque supervisor allocation patterns across departments. Final-year students face unstructured review timelines, while project coordinators encounter difficulties monitoring the overall departmental research lifecycle. The developed E-Supervision platform directly mitigates these institutional challenges by providing a centralized digital portal that enables students to submit chapter drafts sequentially, allows the 13 SCIT lecturers to seamlessly attach inline review commentary and track their active workloads, and empowers the 2 Heads of Department to balance faculty allocations via an analytical backend.

Furthermore, this study emphasizes the critical importance of adopting integrated academic workflow automation systems within higher education structures. By simplifying the project tracking lifecycle and maintaining strict data integrity, the platform contributes to a more efficient, accountable, and collaborative research environment within the School of Computing and Information Technology.

## **5.1 Conclusion**

The E-Supervision platform represents a structurally sound, highly effective, and innovative digital solution for advancing undergraduate research management within the University of Kigali. Its implementation significantly enhances the visibility of project milestone tracking, structures the feedback loop between faculty supervisors and student researchers, and establishes a data-driven model for balancing department-wide supervision assignments.

The software solution successfully eliminates the administrative vulnerabilities associated with isolated or manual record-keeping by centralizing all active project draft variants, formal meeting timelines, and allocation matrices within a secure, 3NF-normalized PostgreSQL relational database. By utilizing role-based access control paradigms, the application maintains strict operational security boundaries, improves departmental transparency, and supports strategic decision-making for coordinators through real-time workload audit streams.

Ultimately, the study demonstrates that embracing automated workflow architecture is essential for modern, data-dense academic environments. The successful design, programmatic

implementation, and subsequent validation testing of this platform confirm that the engineered web application effectively addresses the historical coordination bottlenecks identified in earlier chapters, thereby driving academic excellence, institutional efficiency, and scalable resource utilization across the SCIT School.

## **5.2 Recommendations**

Based on the systemic findings, design outcomes, and feedback gathered during the testing phases of this study, the following professional recommendations are proposed:

### **1. Recommendation to University Management and the School Administration**

**Adopt the Platform Institutionally:** The School of Computing and Information Technology (SCIT) should officially transition from the manual, email-based tracking process to this centralized digital platform. This will eliminate document loss, speed up feedback times, and ensure students don't miss graduation timelines.

**Implement Workload Balancing Rules:** The department should use the system's administrative dashboard to strictly enforce maximum supervision limits per lecturer. This prevents faculty burnout and ensures every final-year student gets equal attention.

### **2. Recommendation to Academic Staff and Final-Year Students**

**Conduct Training Workshops:** The university should organize brief orientation sessions at the start of the academic year. These workshops will teach students how to upload drafts and track milestones, and show faculty supervisors how to log correction statuses and reviews smoothly.

**Enforce System-Only Communication:** Supervisors and students should be required to communicate feedback strictly through the platform rather than via personal WhatsApp or external emails. This keeps an official, unalterable historical log of all project revisions.

### **3. Recommendation to the University IT Infrastructure Department**

**Provide Reliable Server Hosting and Internet:** The IT department should host the FastAPI and PostgreSQL application infrastructure on a secure institutional server with a high-speed campus internet connection. This ensures the platform stays online and responsive during peak submission deadlines.

**Automate System Backups:** Technical administrators must set up automated, daily database backups for the PostgreSQL system. This guarantees that student thesis data and faculty evaluation logs are never lost in the event of a system power failure.

### 5.3 Areas for further research

Future development iterations of the E-Supervision platform should incorporate:

**Smart Supervisor Assignment:** Adding smart tools to the system that can automatically pair students with the right supervisor based on the student's research topic and how many students the lecturer is already guiding.

**Plagiarism Checker Integration:** Connecting the platform to online tools (like Turnitin) so that when a student uploads a chapter, the system automatically checks it for copying or cheating.

**Mobile App Support:** Building a mobile app version of the platform so that students and supervisors can get instant phone alerts about new feedback, text messages, or meeting updates.

**Live Document Editing:** Creating a shared online workspace within the platform where a student and a supervisor can view, edit, and talk about a research document together at the exact same time.

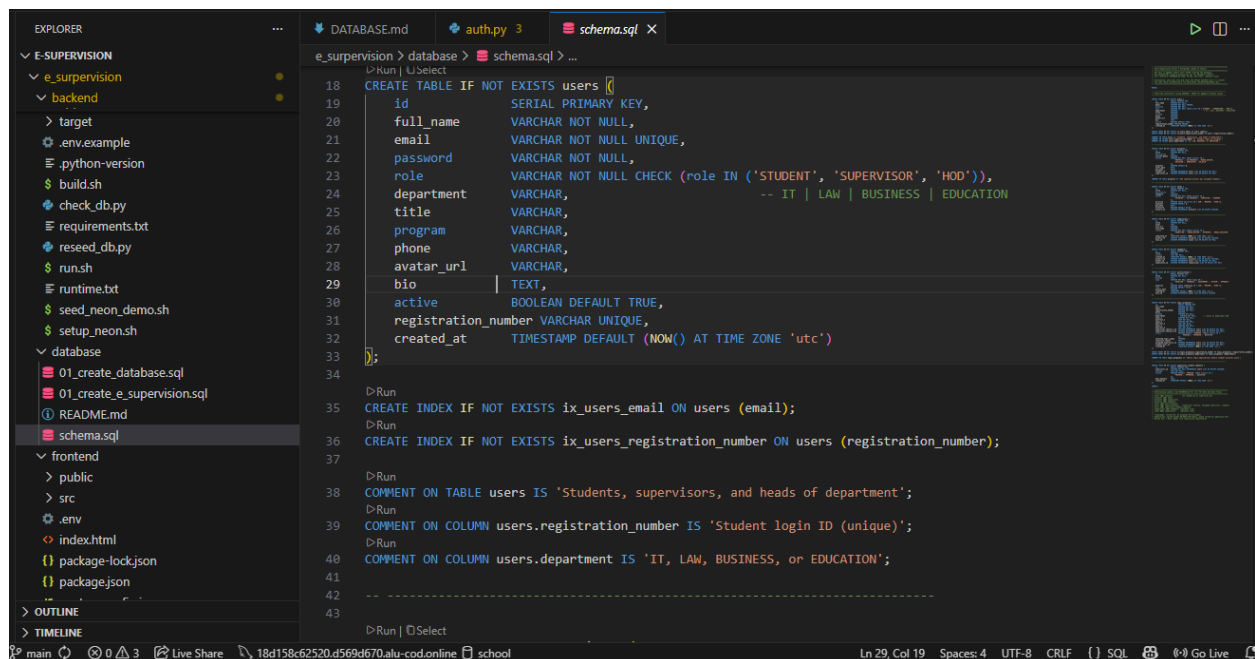
**Stronger Security Protections:** Using advanced security and encryption tools to protect student research ideas and university data from being stolen or hacked.

## APPENDICES

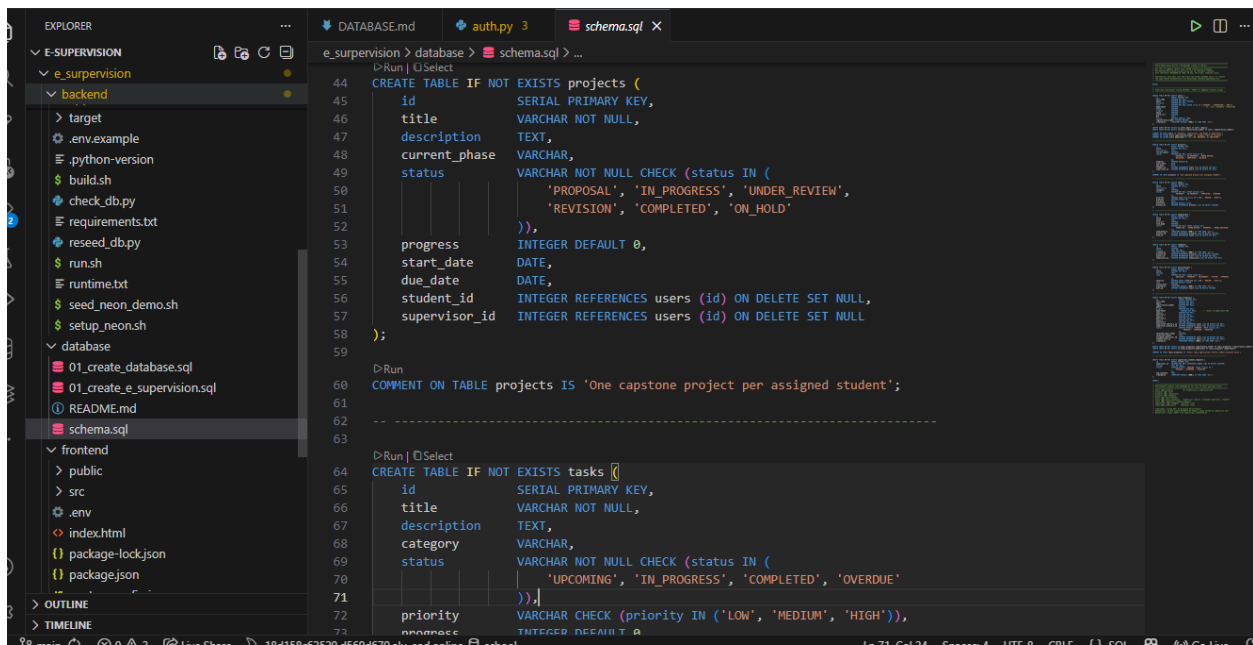
### APPENDIX A: SOURCE CODE

This appendix presents selected architectural source code snippets used in the development of the E-Supervisor Platform. The code demonstrates the system's core functional implementations, including secure role-based user authentication, automated student-supervisor allocation algorithms, electronic milestone document submission, digital supervisor feedback mechanisms, and the underlying relational database integration. These snippets highlight the application of Python with FastAPI for backend routing, React with TypeScript for type-safe frontend components, and PostgreSQL for robust data management.

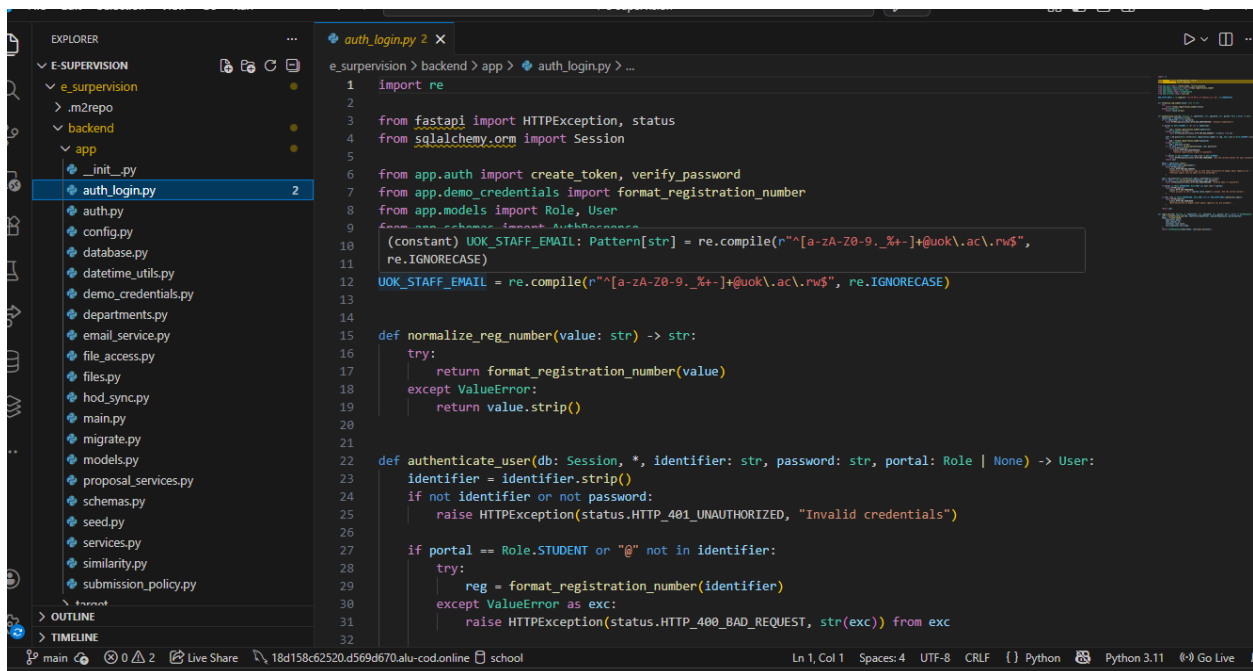
## Relational Database Schema Script



```
18 CREATE TABLE IF NOT EXISTS users
19   id SERIAL PRIMARY KEY,
20   full_name VARCHAR NOT NULL,
21   email VARCHAR NOT NULL UNIQUE,
22   password VARCHAR NOT NULL,
23   role VARCHAR NOT NULL CHECK (role IN ('STUDENT', 'SUPERVISOR', 'HOD')),
24   department VARCHAR,
25   title VARCHAR,
26   program VARCHAR,
27   phone VARCHAR,
28   avatar_url VARCHAR,
29   bio TEXT,
30   active BOOLEAN DEFAULT TRUE,
31   registration_number VARCHAR UNIQUE,
32   created_at TIMESTAMP DEFAULT (NOW() AT TIME ZONE 'utc')
33 ;
34
35 CREATE INDEX IF NOT EXISTS ix_users_email ON users (email);
36 CREATE INDEX IF NOT EXISTS ix_users_registration_number ON users (registration_number);
37
38 COMMENT ON TABLE users IS 'Students, supervisors, and heads of department';
39 COMMENT ON COLUMN users.registration_number IS 'Student login ID (unique)';
40 COMMENT ON COLUMN users.department IS 'IT, LAW, BUSINESS, or EDUCATION';
41
42
43
```



## User Authentication Code





```
e_surveillance > backend > app > auth_login.py > ...
22 def authenticate_user(db: Session, *, identifier: str, password: str, portal: Role | None) -> User:
26
27
28 if portal == Role.STUDENT or "@" not in identifier:
29     try:
30         reg = format_registration_number(identifier)
31     except ValueError as exc:
32         raise HTTPException(status.HTTP_400_BAD_REQUEST, str(exc)) from exc
33
34     user = db.query(User).filter(User.registration_number == reg, User.role == Role.STUDENT).first()
35     try:
36         pwd = format_registration_number(password)
37     except ValueError:
38         pwd = password.strip()
39     if not user or not verify_password(pwd, user.password):
40         raise HTTPException(
41             status.HTTP_401_UNAUTHORIZED,
42             "Invalid registration number or password",
43         )
44     if portal == Role.STUDENT and user.role != Role.STUDENT:
45         raise HTTPException(status.HTTP_403_FORBIDDEN, "Use the correct portal for your account")
46     return user
47
48 email = identifier.lower()
49 if not UOK_STAFF_EMAIL.match(email):
50     raise HTTPException(
51         status.HTTP_400_BAD_REQUEST,
52         "Supervisors and HOD must sign in with their University of Kigali email (@uok.ac.rw). "
53         "Personal emails such as Gmail are not permitted.",
54     )
55
56 user = db.query(User).filter(User.email.ilike(email)).first()
57 if not user or not verify_password(password, user.password):
```

## Supervisor Allocation Code

```
e_surveillance > backend > app > hod_sync.py > ...
14
15 SUPERVISOR_DEPTS: dict[str, Department] = {
16     "jean.bosco@uok.ac.rw": Department.IT,
17     "sarah.mukandoli@uok.ac.rw": Department.IT,
18     "eric.habimana@uok.ac.rw": Department.IT,
19     "it.nshuti@uok.ac.rw": Department.IT,
20     "finance.uwase@uok.ac.rw": Department.BUSINESS,
21     "business.mugisha@uok.ac.rw": Department.BUSINESS,
22     "law.kamanzi@uok.ac.rw": Department.LAW,
23     "law.uwase@uok.ac.rw": Department.LAW,
24     "education.niyonsenga@uok.ac.rw": Department.EDUCATION,
25     "education.mutoni@uok.ac.rw": Department.EDUCATION,
26 }
27
28 EXTRA_SUPERVISORS: list[tuple[str, str, str, Department]] = [
29     ("Dr. Claire Uwase", "finance.uwase@uok.ac.rw", "Finance & Accounting", Department.BUSINESS),
30     ("Dr. Kevin Mugisha", "business.mugisha@uok.ac.rw", "Marketing & Management", Department.BUSINESS),
31     ("Dr. Emmanuel Kamanzi", "law.kamanzi@uok.ac.rw", "Commercial Law", Department.LAW),
32     ("Dr. Grace Uwase", "law.uwase@uok.ac.rw", "Constitutional Law", Department.LAW),
33     ("Dr. Diane Niyonsenga", "education.niyonsenga@uok.ac.rw", "Early Childhood Education", Department.EDUC),
34     ("Dr. Patrick Mutoni", "education.mutoni@uok.ac.rw", "Curriculum Studies", Department.EDUCATION),
35     ("Dr. Emmanuel Nshuti", "it.nshuti@uok.ac.rw", "Data Science", Department.IT),
36 ]
37
38
39 def sync_department_structure(db) -> None:
40     from app.models import TopicProposal
41     from app.departments import department_for_program
42
43     changed = False
44
45     for name, email, dept in HOD_SYNC:
```

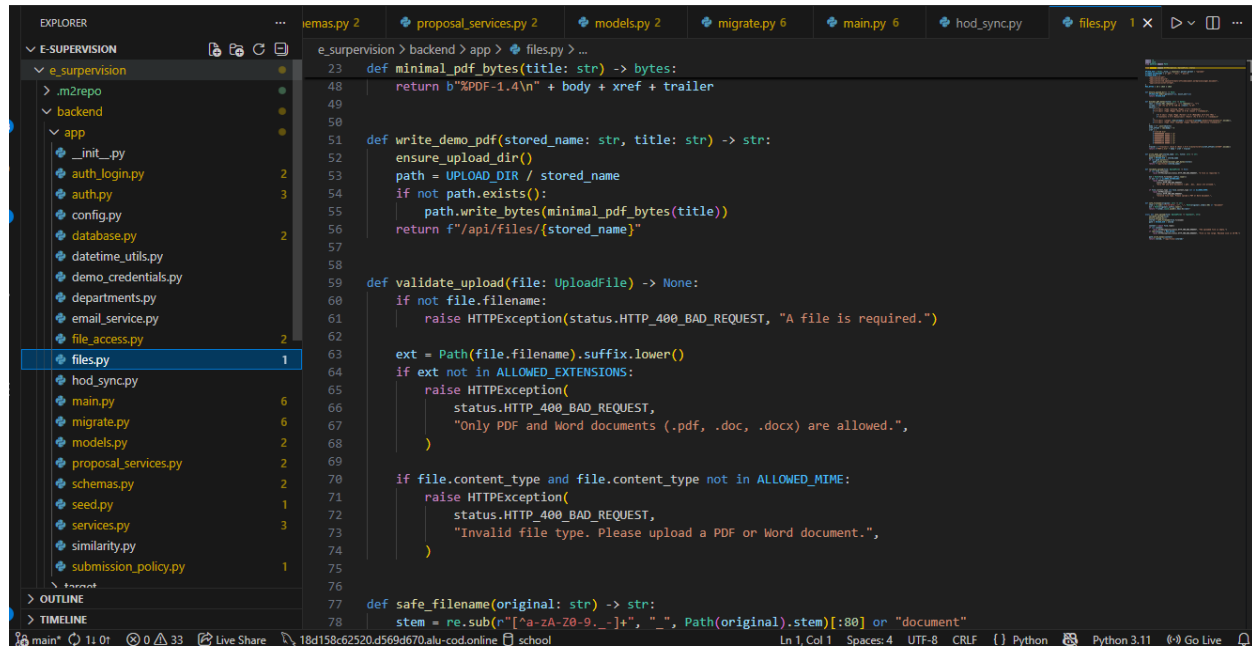
## Document Submission Code

```
e_surveillance > backend > app > submission_policy.py > ...
1 from datetime import datetime, timezone
2
3 from fastapi import HTTPException, status
4 from zoneinfo import ZoneInfo
5
6 from app.config import settings
7 from app.datetime_utils import ensure_utc, to_kigali
8
9
10 def submission_now() -> datetime:
11     return datetime.now(ZoneInfo(settings.submission_timezone))
12
13
14 def validate_submission_window() -> None:
15     if not settings.submission_window_enabled:
16         return
17
18     now = submission_now()
19     start = settings.submission_window_start_hour
20     end = settings.submission_window_end_hour
21     if start <= now.hour < end:
22         return
23
24     raise HTTPException(
25         status.HTTP_400_BAD_REQUEST,
26         f"Submissions are accepted between {start:02d}:00 and {end:02d}:00 "
27         f"({settings.submission_timezone}). Current time: {now.strftime('%H:%M')}. "
28         "Morning submissions are reviewed before afternoon ones.",
29     )
30
31
32 def submission_priority_key(submitted_at: datetime | None) -> tuple[int, datetime]:
```

## Milestone Feedback Code

```
e_surveillance > backend > app > services.py > ...
474
475
476 def review_submission(db: Session, supervisor: User, submission_id: int, *, status: str, feedback: str | None):
477     submission = (
478         db.query(Submission)
479         .options(joinedload(Submission.project).joinedload(Project.student))
480         .filter(Submission.id == submission_id)
481         .first()
482     )
483     if not submission:
484         raise HTTPException(status.HTTP_404_NOT_FOUND, "Submission not found")
485     if not submission.project or submission.project.supervisor_id != supervisor.id:
486         raise HTTPException(status.HTTP_403_FORBIDDEN, "You are not the supervisor for this submission")
487
488     try:
489         submission.status = SubmissionStatus(status)
490     except ValueError:
491         raise HTTPException(status.HTTP_400_BAD_REQUEST, f"Invalid submission status: {status}")
492
493     if feedback and feedback.strip():
494         db.add(
495             Feedback(
496                 title=f"Review: {submission.title}",
497                 content=feedback,
498                 project_id=submission.project_id,
499                 author_id=supervisor.id,
500                 submission_id=submission.id,
501             )
502         )
503
504     student = submission.project.student
505     if student:
```

## Upload Milestone Code



```
23 def minimal_pdf_bytes(title: str) -> bytes:
48     return b"%PDF-1.4\n" + body + xref + trailer
49
50
51 def write_demo_pdf(stored_name: str, title: str) -> str:
52     ensure_upload_dir()
53     path = UPLOAD_DIR / stored_name
54     if not path.exists():
55         path.write_bytes(minimal_pdf_bytes(title))
56     return f"/api/files/{stored_name}"
57
58
59 def validate_upload(file: UploadFile) -> None:
60     if not file.filename:
61         raise HTTPException(status.HTTP_400_BAD_REQUEST, "A file is required.")
62
63     ext = Path(file.filename).suffix.lower()
64     if ext not in ALLOWED_EXTENSIONS:
65         raise HTTPException(
66             status.HTTP_400_BAD_REQUEST,
67             "Only PDF and Word documents (.pdf, .doc, .docx) are allowed.",
68         )
69
70     if file.content_type and file.content_type not in ALLOWED_MIME:
71         raise HTTPException(
72             status.HTTP_400_BAD_REQUEST,
73             "Invalid file type. Please upload a PDF or Word document.",
74         )
75
76
77 def safe_filename(original: str) -> str:
78     stem = re.sub(r"[^a-zA-Z0-9._-]+", "_", Path(original).stem)[:80] or "document"
```

## APPENDIX B: ADDITIONAL SYSTEM SCREENSHOTS

This appendix contains additional screenshots of the E-Supervisor Platform user interfaces and system dashboards that were not included in the main chapters. These visual references illustrate the operational layout of the web application, detailing the distinct workspace dashboards developed for final-year students, faculty supervisors, and Heads of Departments.

### Student Login Interface



UNIVERSITY  
of KIGALI

Student Portal



# Student Portal

Track your capstone project, submit work, and receive supervisor feedback.

© 2026 University of Kigali. All rights reserved.



## Student Sign in

Sign in with your student credentials.

Registration Number

# 202305000078

12-digit registration number (YYYYTTNNNNNN). Your password is the same as your reg number.

Password

••••••••

→ Sign in to Student Portal

Student Progress Dashboard



UNIVERSITY  
of KIGALI

Student Portal

Personal Profile

SUPERVISION

My Project

Tasks & Milestones

Submissions

Feedback

EXAMINATION

Progress Tracker

Notifications 2

Sign out

Capstone E-Supervision Portal  
Student Portal

 2 **Aggie Moraa**  
BSc Software Engineering **AM**

Welcome back, Aggie

Here's how your capstone project is progressing.

New Submission

Overall Progress

68%

Days Remaining

16

Due 25 Jul 2026

Current Phase

Chapter 3

In Progress

Next Milestone

Final Design Submission

Due 25 Jul 2026

Capstone Project

AI-Driven Academic Supervision Platform

A web platform streamlining capstone supervision between students, supervisors, and HODs.

Started

3 Apr 2026

Due

25 Jul 2026

Phase

Chapter 3

Upcoming Tasks

Literature Review & Analysis

Due 4 Jul 2026

In Progress

My Supervisor

 **Dr. Jean Bosco**  
Dr.

jean.bosco@uok.ac.rw

+250 788 200 000

View Feedback

Recent Feedback

22°

Search

ENG UK

01:52  
10/07/2026

Document Submission Form

UNIVERSITY  
KIGALI

ent Portal

Personal Profile

UPERVISION

My Project

Tasks & Milestones

Submissions

Feedback

AMINATION

Progress Tracker

Notifications 2

Sign out

Capstone E-Supervision Portal

Student Portal

2

Aggie Moraa  
BSc Software Engineering

AM

Submissions

Upload PDF or Word documents only (.pdf, .doc, .docx) — max 10 MB.

✕ Cancel

New Submission

Upload a deliverable for supervisor review

Title

data analysis

Notes for supervisor

finished the chapter

Document (PDF or Word)

Choose File frame c.docx

Selected: frame c.docx (28.8 KB)

Submit for Review

## Supervisor Login Interface

Supervisor Portal

Supervisor

Portal

Review submissions, monitor student progress, and manage your supervision workload.

@ 2028 University of Kigali. All rights reserved.

Supervisor Sign in

Sign in with your supervisor credentials.

University Email

jean.bosco@uok.ac.rw

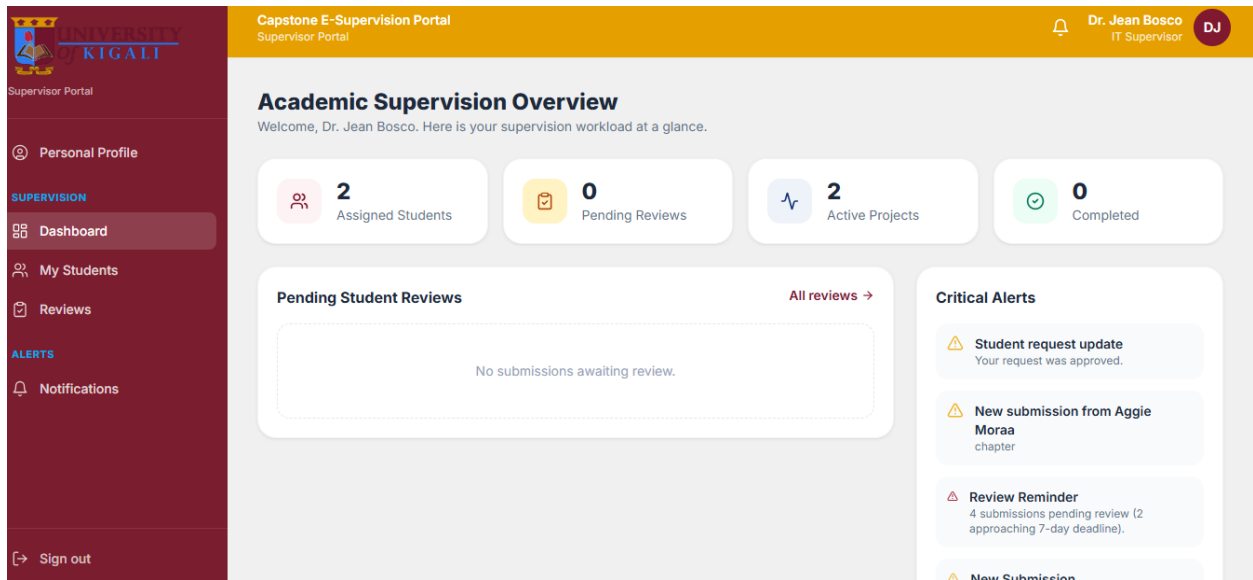
Use your official University of Kigali email (@uok.ac.rw) only — not Gmail or other providers.

Password

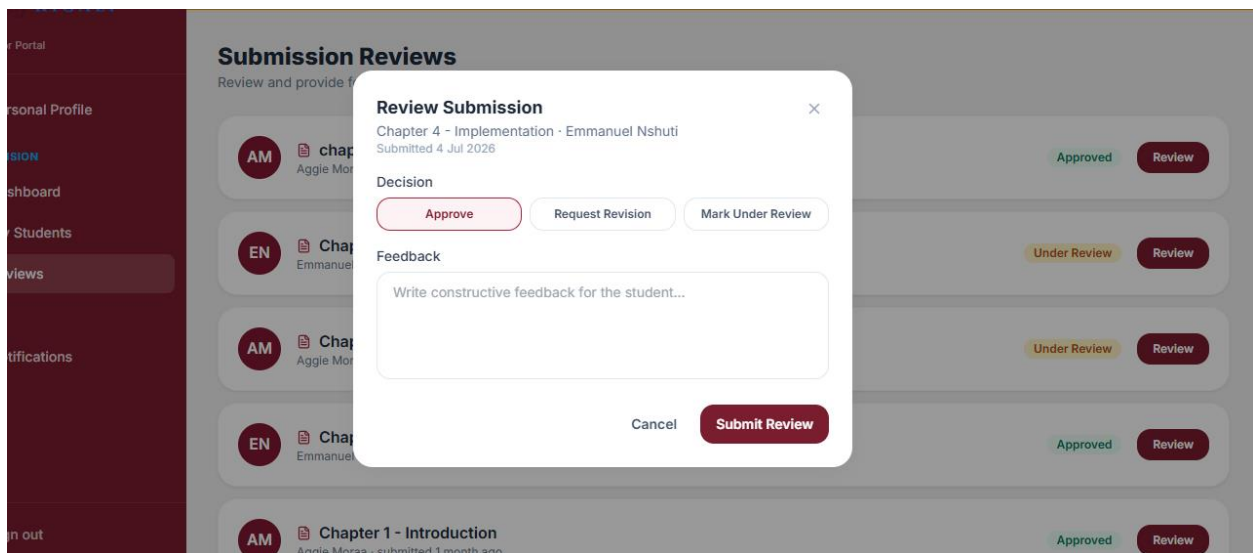
.....

→ Sign in to Supervisor Portal

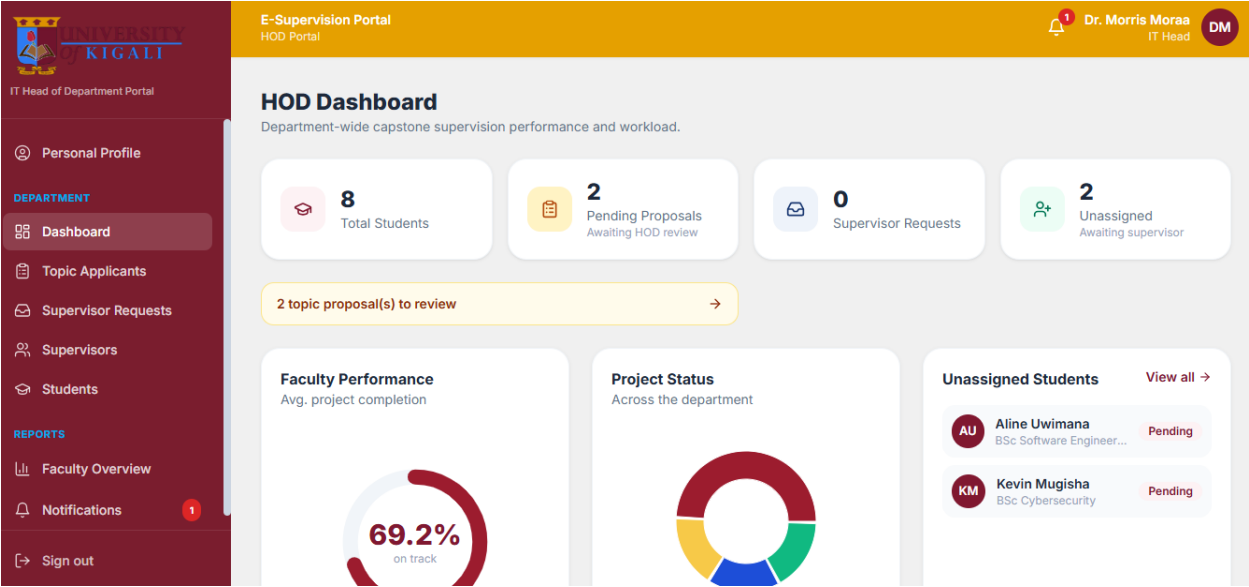
## Supervisor Dashboard



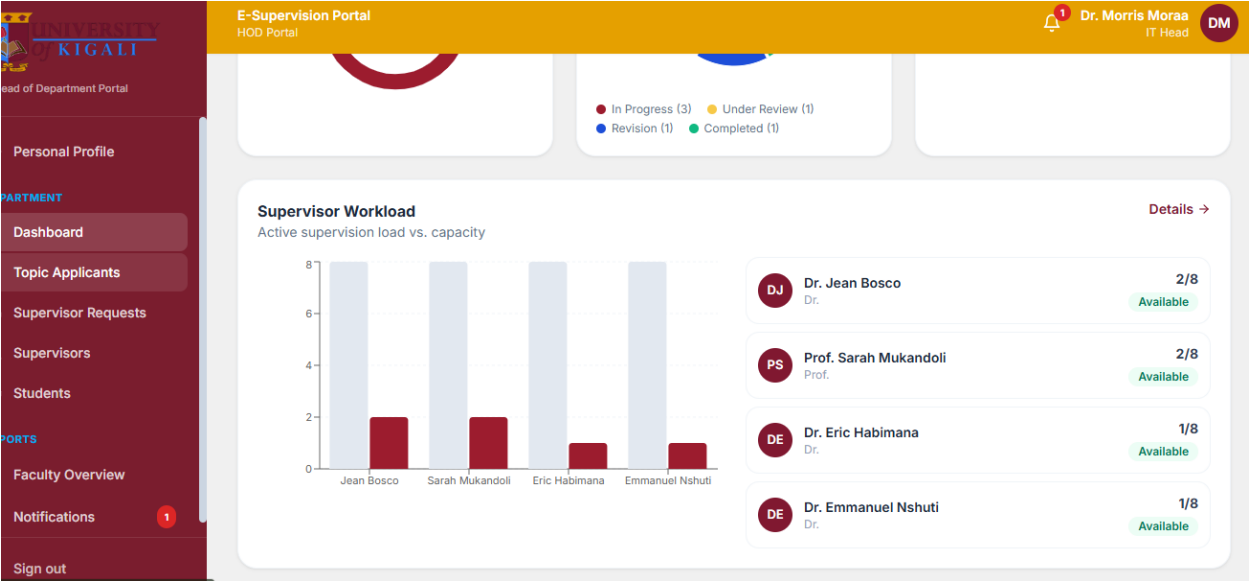
## Evaluation & Feedback Workspace



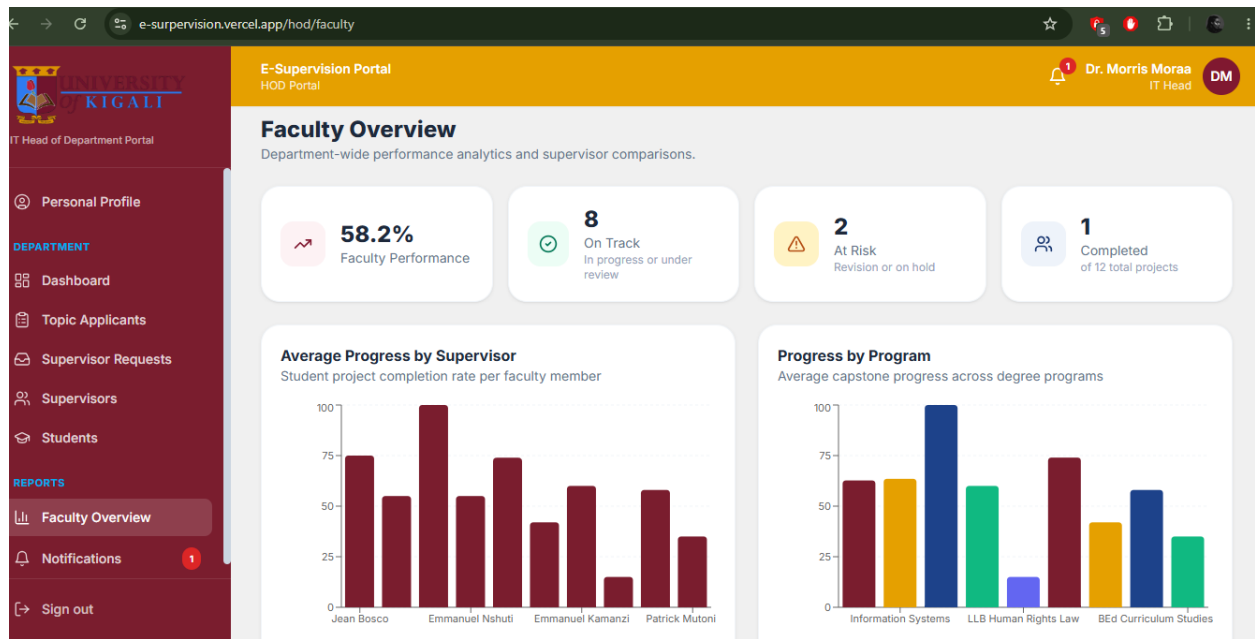
## HOD Dashboard Overview



## Supervisor Allocation Interface



## Departmental Progress Reports



| Supervisor                                     | Students | Avg. Progress | Completed | At Risk |
|------------------------------------------------|----------|---------------|-----------|---------|
| <a href="#">Dr. Jean Bosco</a><br>Dr.          | 2        | 75%           | 0         | 0       |
| <a href="#">Prof. Sarah Mukandoli</a><br>Prof. | 2        | 55%           | 0         | 1       |
| <a href="#">Dr. Eric Habimana</a><br>Dr.       | 1        | 100%          | 1         | 0       |
| <a href="#">Dr. Emmanuel Nshuti</a><br>Dr.     | 1        | 55%           | 0         | 0       |
| <a href="#">Dr. Claire Uwase</a><br>Dr.        | 1        | 74%           | 0         | 0       |
| <a href="#">Dr. Kevin Mugisha</a><br>Dr.       | 1        | 42%           | 0         | 0       |
| <a href="#">Dr. Emmanuel Kamanzi</a><br>Dr.    | 1        | 55%           | 0         | 0       |

## APPENDIX C: DATABASE STRUCTURE

This appendix presents the database structure used in the E-Supervisor Platform. The system uses a relational database management system (PostgreSQL) to store, manage, and secure system data.

### Users Table:

CREATE TABLE IF NOT EXISTS users (

id SERIAL PRIMARY KEY,

full\_name VARCHAR NOT NULL,



```

email      VARCHAR NOT NULL UNIQUE,

password   VARCHAR NOT NULL,

role       VARCHAR NOT NULL CHECK (role IN ('STUDENT', 'SUPERVISOR', 'HOD')),

department VARCHAR,                -- IT | LAW | BUSINESS | EDUCATION

title      VARCHAR,

program    VARCHAR,

phone      VARCHAR,

avatar_url VARCHAR,

bio        TEXT,

active     BOOLEAN DEFAULT TRUE,

registration_number VARCHAR UNIQUE,

created_at  TIMESTAMP DEFAULT (NOW() AT TIME ZONE 'utc')

);

```

### **projects Table:**

```

CREATE TABLE IF NOT EXISTS projects (

    id          SERIAL PRIMARY KEY,

    title       VARCHAR NOT NULL,

    description  TEXT,

    current_phase VARCHAR,

    status      VARCHAR NOT NULL CHECK (status IN (

        'PROPOSAL', 'IN_PROGRESS', 'UNDER_REVIEW',

        'REVISION', 'COMPLETED', 'ON_HOLD'

    )),

```

```
progress    INTEGER DEFAULT 0,
start_date  DATE,
due_date    DATE,
student_id  INTEGER REFERENCES users (id) ON DELETE SET NULL,
supervisor_id  INTEGER REFERENCES users (id) ON DELETE SET NULL
);
```

### **Tasks Table:**

```
CREATE TABLE IF NOT EXISTS tasks (
    id          SERIAL PRIMARY KEY,
    title       VARCHAR NOT NULL,
    description  TEXT,
    category    VARCHAR,
    status      VARCHAR NOT NULL CHECK (status IN (
        'UPCOMING', 'IN_PROGRESS', 'COMPLETED', 'OVERDUE'
    )),
    priority    VARCHAR CHECK (priority IN ('LOW', 'MEDIUM', 'HIGH')),
    progress    INTEGER DEFAULT 0,
    due_date    DATE,
    milestone   BOOLEAN DEFAULT FALSE,
    project_id  INTEGER REFERENCES projects (id) ON DELETE CASCADE
);
```

### **Submissions Table:**

```
CREATE TABLE IF NOT EXISTS submissions (
```

```

id          SERIAL PRIMARY KEY,

title       VARCHAR NOT NULL,

notes       TEXT,

file_url    VARCHAR,

file_name   VARCHAR,

status      VARCHAR NOT NULL CHECK (status IN (

            'SUBMITTED', 'UNDER_REVIEW', 'APPROVED', 'NEEDS_REVISION'

            )),

submitted_at  TIMESTAMP DEFAULT (NOW() AT TIME ZONE 'utc'),

project_id   INTEGER REFERENCES projects (id) ON DELETE CASCADE,

task_id      INTEGER REFERENCES tasks (id) ON DELETE SET NULL

);

```

### **Feedback Table:**

```

CREATE TABLE IF NOT EXISTS feedback (

id          SERIAL PRIMARY KEY,

title       VARCHAR,

content     TEXT NOT NULL,

created_at  TIMESTAMP DEFAULT (NOW() AT TIME ZONE 'utc'),

project_id  INTEGER REFERENCES projects (id) ON DELETE CASCADE,

author_id   INTEGER REFERENCES users (id) ON DELETE SET NULL,

submission_id  INTEGER REFERENCES submissions (id) ON DELETE SET NULL

);

```

### **Notifications Table:**

```

CREATE TABLE IF NOT EXISTS notifications (

    id          SERIAL PRIMARY KEY,

    title       VARCHAR NOT NULL,

    message     TEXT,

    type        VARCHAR NOT NULL CHECK (type IN (

                'DEADLINE', 'FEEDBACK', 'ASSIGNMENT', 'SYSTEM', 'APPROVAL'

            )),

    severity    VARCHAR CHECK (severity IN ('LOW', 'MEDIUM', 'HIGH')),

    read        BOOLEAN DEFAULT FALSE,

    action_path VARCHAR,

    created_at  TIMESTAMP DEFAULT (NOW() AT TIME ZONE 'utc'),

    user_id     INTEGER REFERENCES users (id) ON DELETE CASCADE

);

```

### **topic\_proposals Table:**

```

CREATE TABLE IF NOT EXISTS topic_proposals (

    id          SERIAL PRIMARY KEY,

    full_name    VARCHAR NOT NULL,

    email        VARCHAR NOT NULL,

    registration_number VARCHAR NOT NULL,

    phone        VARCHAR,

    program      VARCHAR NOT NULL,

    department   VARCHAR NOT NULL,    -- routes to department HOD

    topic_1      VARCHAR NOT NULL,

```

```

abstract_1      TEXT NOT NULL,
topic_2         VARCHAR NOT NULL,
abstract_2      TEXT NOT NULL,
topic_3         VARCHAR NOT NULL,
abstract_3      TEXT NOT NULL,
supervisor_choice_1_id INTEGER REFERENCES users (id) ON DELETE SET NULL,
supervisor_choice_2_id INTEGER REFERENCES users (id) ON DELETE SET NULL,
status          VARCHAR DEFAULT 'PENDING' CHECK (status IN (
                'PENDING', 'APPROVED', 'REJECTED'
                )),
selected_topic_index INTEGER,
rejection_reason TEXT,
assigned_supervisor_id INTEGER REFERENCES users (id) ON DELETE SET NULL,
student_user_id  INTEGER REFERENCES users (id) ON DELETE SET NULL,
created_at       TIMESTAMP DEFAULT (NOW() AT TIME ZONE 'utc')
);

```

#### **supervisor\_student\_requests Table:**

```

CREATE TABLE IF NOT EXISTS supervisor_student_requests (
    id          SERIAL PRIMARY KEY,
    supervisor_id INTEGER NOT NULL REFERENCES users (id) ON DELETE CASCADE,
    message     TEXT NOT NULL,
    status      VARCHAR DEFAULT 'PENDING' CHECK (status IN (
                'PENDING', 'APPROVED', 'REJECTED'

```

```

)),

hod_response TEXT,

created_at TIMESTAMP DEFAULT (NOW() AT TIME ZONE 'utc')

);

```

## APPENDIX D: TESTING EVIDENCE

This appendix presents testing evidence used to verify the functionality, reliability, and performance of the E-Supervisor Platform.

### D.1 Unit Testing (Testing Individual Pieces)

| Module Tested              | Test Input                                    | Expected Result                      | Actual Result                 | Status |
|----------------------------|-----------------------------------------------|--------------------------------------|-------------------------------|--------|
| <b>User Login</b>          | Valid student ID or lecturer email            | Log in successfully                  | Logged in successfully        | Passed |
| <b>Assign Supervisor</b>   | HOD links a student to a lecturer             | The lecturer's student list grows    | List updated correctly        | Passed |
| <b>Upload Document</b>     | Student uploads Chapter 1 PDF                 | File saves with the exact time       | File saved with time recorded | Passed |
| <b>Lecturer Feedback</b>   | Lecturer types a comment and clicks "Approve" | System moves student to next chapter | Student moved to next stage   | Passed |
| <b>Database Connection</b> | System requests data                          | Smooth connection to PostgreSQL      | Connected with no issues      | Passed |

## D.2 Integration Testing (Testing How Pieces Work Together)

| Components Integrated | Expected Result                                    | Actual Result                    | Status |
|-----------------------|----------------------------------------------------|----------------------------------|--------|
| Frontend & Backend    | The screens talk to the background server smoothly | Communication works perfectly    | Passed |
| Backend & Database    | Server saves data into PostgreSQL correctly        | Data stored with no errors       | Passed |
| Screens & Security    | Students are blocked from seeing HOD admin screens | System blocks unauthorized users | Passed |

## D.3 Functional Testing (Testing Main Features)

| Function Tested   | Expected Output                              | Actual Output                                                | Status |
|-------------------|----------------------------------------------|--------------------------------------------------------------|--------|
| Role-Based Access | Show the right screen to the right user type | Students see student screens, lecturers see lecturer screens | Passed |

| <b>Function Tested</b>     | <b>Expected Output</b>                                          | <b>Actual Output</b>                                    | <b>Status</b> |
|----------------------------|-----------------------------------------------------------------|---------------------------------------------------------|---------------|
| <b>Workload Warning</b>    | Block HOD if they try to give too many students to one lecturer | System blocks the assignment and shows an error warning | Passed        |
| <b>Progress Bar Update</b> | Progress bar moves when a chapter gets approved                 | Screen updates instantly when approved                  | Passed        |

#### **D.4 Acceptance Testing (Testing User Satisfaction)**

| <b>User Group</b>                 | <b>User Feedback</b>                                                     | <b>Result</b>   |
|-----------------------------------|--------------------------------------------------------------------------|-----------------|
| <b>Students</b>                   | The system makes it very easy to track milestones and upload chapters.   | <b>Accepted</b> |
| <b>Faculty Supervisors</b>        | Seeing all assigned students in one clean list saves a lot of time.      | <b>Accepted</b> |
| <b>Heads of Departments (HOD)</b> | Managing lecturer workloads and checking student progress is simple now. | <b>Accepted</b> |

#### **CONCLUSION OF TESTING**

In conclusion, the testing phase proved that the E-Supervisor Platform works exactly as intended. The functional tests showed that core features such as secure role-based login, student-supervisor allocation, digital draft submissions, and milestone tracking execute correctly without errors. At the same time, non-functional testing confirmed that the system is fast, highly reliable, and keeps



user data secure. Because the application handles high workloads smoothly and maintains strict data accuracy within the PostgreSQL database, it stands as an efficient, stable solution ready to support the academic tracking needs.

## **QUESTIONNAIRE FOR RESPONDENTS**

Dear Respondent,

I am a final-year student at the University of Kigali pursuing a Bachelor of Business Information Technology (BBIT). I am conducting a research study entitled "**E-Supervisor Platform.**" The information you provide will be used strictly for academic purposes and will be treated with the utmost confidentiality. Your participation is highly appreciated and will contribute significantly to the success of this research project.

Thank you for your cooperation.

Please tick (✓) the appropriate answer.

### **Section A: General Information**

**1. What is your current role within the School of Computing and Information Technology (SCIT)?**

- ☐ Final-Year Student
- ☐ Faculty Supervisor / Lecturer
- ☐ Head of Department (HOD) / Program Coordinator

**2. How long have you been involved in managing or undertaking undergraduate thesis research projects?**

- ☐ Less than 1 year
- ☐ 1–2 years

☐ 2–3 years

☐ More than 3 years

### **Section B: Current Manual Supervision System Experience**

#### **3. How satisfied are you with the current manual or email-based project tracking process?**

☐ Very Satisfied

☐ Satisfied

☐ Neutral

☐ Dissatisfied

☐ Very Dissatisfied

#### **4. How easy is it to track project milestones and view older document versions or feedback comments right now?**

☐ Very Easy

☐ Easy

☐ Neutral

☐ Difficult

☐ Very Difficult

#### **5. How often do you experience communication delays or lost files when sending chapter drafts via personal emails or papers?**

☐ Always

☐ Most of the time

- ☐ Sometimes
- ☐ Rarely
- ☐ Never

### **Section C: Current Research Supervision Challenges**

**6. What are the main challenges you face with the current supervision process? (Select all that apply)**

- ☐ Prolonged feedback and long communication turnaround times
- ☐ High risk of document version loss or missing reviewer updates
- ☐ Lack of a real-time progress dashboard to check active milestone statuses
- ☐ Unequal or unbalanced supervisor workload distributions across faculty
- ☐ High difficulty for the department to generate accurate progress reports manually
- ☐ Restrictive schedule boundaries and requirement for physical coordination
- ☐ Other(specify): \_\_\_\_\_

### **Section D: Proposed E-Supervisor Platform Implementation**

**7. Would you utilize a unified, web-based digital platform to manage and track undergraduate thesis projects?**

- ☐ Yes
- ☐ No

**8. Which features of the proposed E-Supervisor Platform would be most useful to you? (Select all that apply)**

- ☐ Secure role-based login using existing registration numbers and institutional emails
- ☐ Centralized electronic document submission for research proposals and chapter drafts
- ☐ A real-time visual progress tracker dashboard displaying completed vs. pending chapters
- ☐ Digital feedback workspace for supervisors to save correction logs and status decisions
- ☐ Administrative dashboard for HODs to allocate students and balance faculty workloads
- ☐ Automated system notifications and deadline warning countdown indicators

**9. Do you agree that moving from manual tracking to an automated E-Supervisor workflow will improve graduation success rates at the University of Kigali?**

- ☐ Strongly Agree
- ☐ Agree
- ☐ Neutral
- ☐ Disagree
- ☐ Strongly Disagree

**10. Would you recommend the implementation of this web platform to other final-year students and academic staff?**

- ☐ Yes
- ☐ No

**11. Provide any additional comments or suggestions for implementing the E-Supervisor Platform:**